

#24 Variable Scope

10/06/2025

- A variable's scope is the part of the program that can access that variable by name.
- Variable scoping rules describe how and where Python finds previously defined variables, and where you can use it.

Recap of Scope chapter within 'Introduction to Programming with Python' by LS (under Functions and Methods)

- Scope of an identifier determines where you can use it.
- Scoping in Python determined where you initialise the identifier.
- Function scope: anything initialised inside a function is only available within that function and any nested functions

Little Surprise

```
def scope_test():  
    if True:  
        foo = "Hello"  
    else:  
        bar = "Goodbye"
```

```
    print(foo)  
    print(bar)
```

```
scope_test()
```

Hello

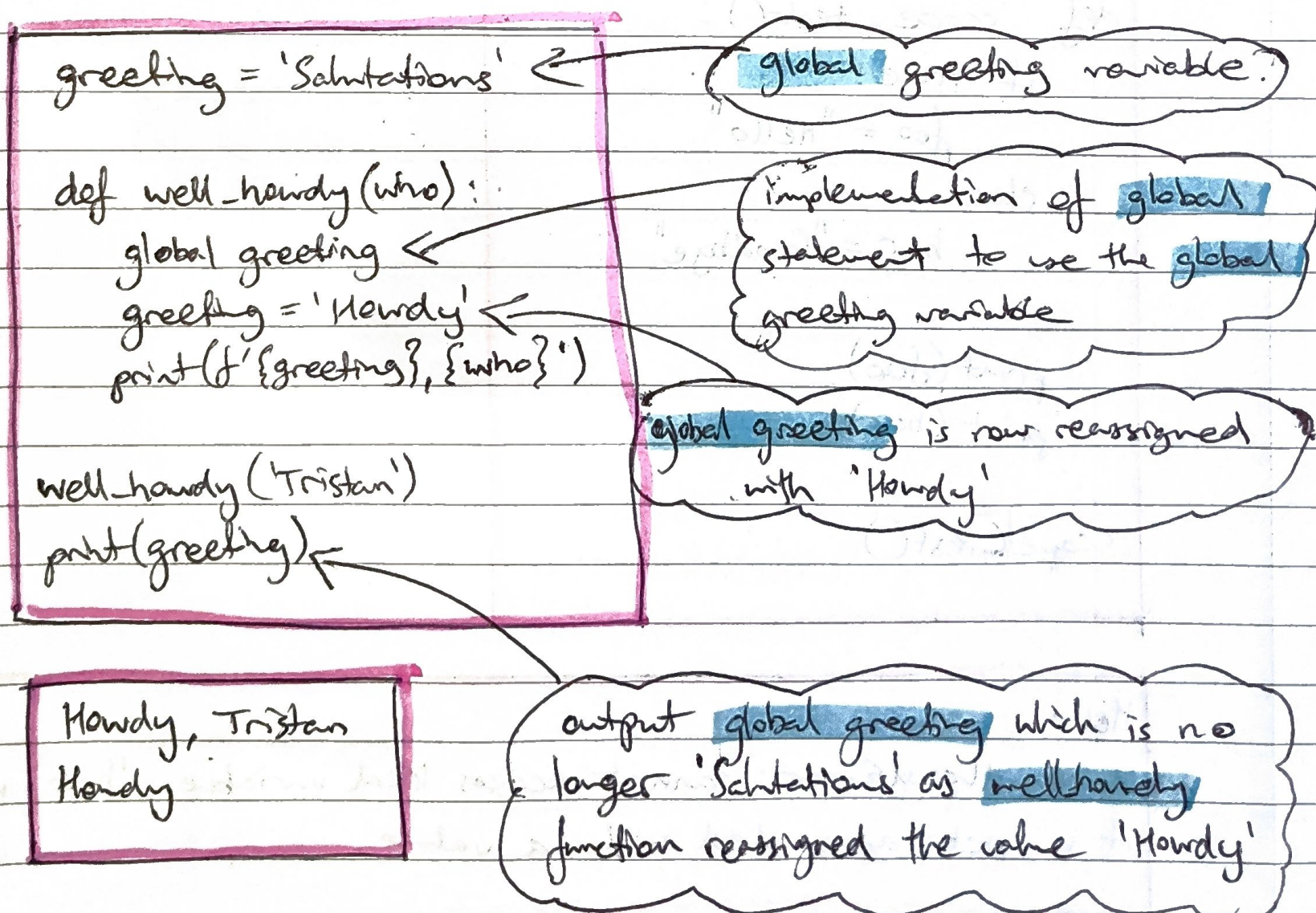
UnboundLocalError: cannot access local variable 'bar' where it is not associated with a value.

10/06/2025

- The `else` block never runs, `bar` is left unassigned, despite both `foo` and `bar` being technically in scope.

Recap of The global and nonlocal statements (under More Stuff) within 'Introduction to Programming with Python' by LS.

- By default, any variable defined outside a function is also available inside that function.
- For Python, all variables assigned a value inside a function are considered `local variables`.
- Variable shadowing is when the same variable name is used within a function as it is outside the function. However, a different value is assigned to it.
- `global` and `nonlocal` statements allow the programmer to override the default behaviour summarized above.



10/06/2025

- You can create a new **global** variable within a function. i.e., a ^{global} variable does not have to be defined outside of a function.

```
def set_pi():  
    global pi  
    pi = 3.1415
```

```
set_pi()  
print(pi) # Output: 3.1415
```

Use global statement within a function to create a new global variable or reassign an existing global variable that was set outside of the function.

- The **nonlocal** statement is used to change a variable in the **outer** scope of a function. i.e., change the variable one level of scope up from the existing.

```
def outer():
```

```
    def inner1():
```

```
        def inner2():
```

```
            nonlocal foo
```

```
            foo = 3
```

```
            print(f"inner2 -> {foo} with id {id(foo)}")
```

```
        foo = 2
```

```
        inner2()
```

```
        print(f"inner1 -> {foo} with id {id(foo)}")
```

```
    foo = 1
```

```
    inner1()
```

```
    print(f"outer -> {foo} with id {id(foo)}")
```

```
outer()
```

```
## print(f"global -> {foo} with id {id(foo)}") # statement removed
```

foo = 3 for inner2 and inner1

10/06/2025

```
inner2 → 3 with id 123456789
inner1 → 3 with id 123456789
outer → with id 987654321
```

Can see `inner2` and `inner1` have same as id.

The `nonlocal` statement in `inner2` allowed for the reassigning of `inner1` `foo` variable with the value assigned to `foo` in `inner2`.

To make `outer` `foo` variable also be reassigned the same value as in `inner2` and `inner1`, need to include a `nonlocal` statement in `inner1`.

```
def outer():
    def inner1():
        def inner2():
            nonlocal foo
            foo = 3
            print(f"inner2 → {foo} with id {id(foo)}")

        nonlocal foo
        foo = 2
        inner2()
        print(f"inner1 → {foo} with id {id(foo)}")

    foo = 1
    inner1()
    print(f"outer → {foo} with id {id(foo)}")
```

foo = 3 for inner2 and inner1

foo = 3 for outer

```
inner2 → 3 with id 123456789
inner1 → 3 with id 123456789
outer → 3 with id 123456789
```

All three are the same

10/06/2025

Global Scope

- Variables initialised outside of any functions have a global scope.
 - Can be accessed from anywhere in your program.

```
name = 'John'
```

```
def greet():  
    print(f"Hello, {name}!")
```

```
greet() # Output: Hello, John!
```

`name` variable is initialised in the `global` scope and is accessed within the `greet` function.

Local Scope

Function scope

- Functions define a new scope for local variables.
- Inner scopes are scopes defined by a function.
- Nested scopes are defined by nested functions
- Variable scope is determined by where it is initialised or declared.

→ In Python, initialising and declaring generally occur at the same time

e.g., `x = 5`

Rule 1: Variables defined in a function are local to that function and cannot be accessed in the outer scope.

10/06/2025

```
def my_func():  
    local_var = 10  
    print(local_var)
```

my_func() # Output: 10

print(local_var) # Raises NameError: name 'local_var' is not defined

Rule 2: Variables that are defined within a function are local unless explicitly marked as **global** or **nonlocal**

```
def my_func():  
    global global_var  
    global_var = 20
```

global keyword used to indicate
global_var being modified is a
global variable

my_func()

print(global_var) # Output: 20

The **nonlocal** statement's use and behaviour is similar to that of **global**. However, it only applies in nested functions

Rule 3: Variables used but not reassigned in a function may be in the outer scope.

```
outer_var = 15
```

```
def my_func():  
    print(outer_var)
```

my_func() # Output: 15

10/06/2025

`outer_var` is used but not reassigned in the `my_func` function. Therefore, Python looks for it in the outer scope and successfully finds the value 15.

Rule 4: Peer scopes do not conflict:

```
1 def func_a():  
2     a = 'hello'  
3     print(a)  
4  
5 def func_b():  
6     print(a) # NameError: name 'a' is not defined  
7  
8 func_a()  
9 func_b()
```

- Executing `print(a)` on line 6 raises an error as `a` is not in `func_b`'s scope.
- `a = 'hello'` defined on line 2 is only within the scope of `func_a`. `func_b` cannot access `func_a` variables.
- Could define a separate `a` variable in `func_b`.
 - The two `a` variables, one in `func_a` and the other in `func_b` would have different local scopes.
 - The two `a` variables would be independent of each other

Rule 5: Nested functions have their own scope

(See Over The Page)

10/06/2025

```
a = 1 # first level variable
```

```
def foo(): # second level  
    b = 2
```

```
def bar(): # third level  
    c = 3
```

```
    print(a) # => 1
```

```
    print(b) # => 2
```

```
    print(c) # => 3
```

```
bar()
```

```
print(a) # => 1
```

```
print(b) # => 2
```

```
print(c) # => NameError: name 'c' is not defined
```

```
foo()
```

Block Scope

There is no block scope for Python. Variables defined within blocks, such as if statements or loops, are accessible outside of those blocks.

```
if True:  
    block_var = 'Hello'
```

```
print(block_var) # Output: Hello
```

block_var initialised within the if statement but still accessible outside of the if statement

Exercises

What will the following snippets of code do and why?

Question 1

```

1 num = 5
2
3 def my_func():
4     print(num) # Output: 5
5
6 my_func()

```

Line 4 will output the global variable value ⁵ assigned to num.

Question 2

```

1 num = 5
2
3 def my_func():
4     num = 10
5
6 my_func()
7 print(num) # Output: 5

```

Lines 3 & 4 do not return `num = 10`. Line 7 prints the value 5 assigned to the global `num` variable.

10/06/2025

Question 3

```
1 num = 5
2
3 def my_func():
4     global num
5     num = 10
6
7 my_func()
8 print(num) # Output: 10
```

Line 8 will output **10** as the global variable on line 1 has been reassigned the value of **10** due to the **global** statement implementation on line 4 of the **my_func** definition.

Question 4

```
1 def outer():
2     outer_var = 'Hello' # <-- defined in outer scope
3
4     def inner():
5         inner_var = 'World' <-- defined in inner scope
6         print(outer_var, inner_var) # Output: Hello World
7
8     inner()
9     # Accesses both outer_var & inner_var.
10 outer()
```

Line 6 will output **Hello World**. As **print** function is called in **inner** function that is inside **outer** function, Both the **inner_var**

10/06/2025

which is initialised inside **inner** function, and the **outer_var** which is initialised one scope up in the **outer** function, are available to print inside the inner function.

NB: Outer function cannot access inner_var.

Question 5

```
1 def my_func():
2     num = 10
3
4 my_func()
5 print(num) # NameError: name 'num' is not defined.
```

An error will occur as `num = 10` has not been initialised globally. Could implement global `num` before line 2 of declare `num = 10` before line 1.

Question 6

```
1 def my_func():
2     x = 15
3
4     def inner_func1():
5         x = 25
6         print("Inner 1:", x) # Output: Inner1: 25
7
8     def inner_func2():
9         print("Inner 2:", x) # Output: Inner2: 15
10
11 inner_func1()
12 inner_func2()
13
14 my_func()
```


10/06/2025

Line 6 would output 25 as x variable is initialized within the `inner_func1` function. Line 9 will output the global variable assigned value declared on line 2 as no local variable, x , is initialized in the `inner_func2` function.

Summary

Understanding VARIABLE SCOPE IS ONE OF THE MOST CHALLENGING AND IMPORTANT ASPECTS OF PROGRAMMING